

Documento de Visão, Arquitetura e Segurança

Sistema de Totem de Autoatendimento para Fast Food

Objetivo do documento: organizar a ideia do sistema de totem de fast food de forma técnica, visual e estratégica, servindo como base para planejamento do backend, definição do MVP, segurança da API, controle de dispositivos, gestão do cardápio e evolução futura do produto.

1. Resumo Executivo

O projeto consiste em um **sistema de autoatendimento para fast foods**, onde o cliente poderá realizar pedidos diretamente em um tablet, totem ou tela touch.

O cliente poderá:

- Visualizar o cardápio.
- Escolher produtos.
- Personalizar itens.
- Selecionar forma de pagamento.
- Receber senha ou comprovante de retirada.
- Acompanhar o status do pedido.

A solução será composta por:

- **Totem do cliente**, responsável pela experiência de autoatendimento.
- **Backend em Java com Spring Boot**, responsável pelas regras de negócio.
- **Banco de dados**, responsável por armazenar produtos, pedidos, pagamentos, usuários e dispositivos.
- **Painel do caixa**, responsável por pagamentos pendentes e confirmações manuais.
- **Painel da cozinha**, responsável pelo acompanhamento e preparo dos pedidos.
- **Painel administrativo**, responsável por cardápio, produtos, categorias, destaques e configurações.
- **Módulo de segurança**, responsável por autenticação, autorização, usuários, dispositivos e permissões.
- **Módulo de pagamento**, preparado para Pix, cartão, dinheiro e integrações futuras com maquininha, SmartPOS ou TEF.

A primeira versão pode utilizar pagamento simulado, permitindo validar o fluxo completo antes de integrar com Pix real, cartão, maquininha ou TEF.

2. Problema que o Sistema Resolve

Em estabelecimentos de fast food, filas no caixa e dependência total de atendentes podem gerar lentidão, erros de comunicação e perda de produtividade.

O sistema busca resolver esses pontos permitindo que o próprio cliente realize o pedido pelo totem, enquanto o restaurante mantém controle sobre pagamento, preparo e retirada.

Benefícios para o cliente

- Autonomia para escolher produtos.
- Visualização clara do cardápio.
- Facilidade para personalizar pedidos.
- Redução de tempo em filas.
- Acompanhamento do pedido por senha.

Benefícios para o restaurante

- Organização do fluxo de pedidos.
 - Redução de sobrecarga no caixa.
 - Separação entre pedido, pagamento e preparo.
 - Controle de pedidos pagos e pendentes.
 - Gestão dinâmica do cardápio sem alteração no código.
 - Possibilidade de expansão para múltiplos dispositivos.
-

3. Visão Geral da Solução

Cliente



Totem / Tablet



Backend Java / Spring Boot



Banco de Dados



+-----+-----+-----+

| | | |

Painel do Caixa Painel da Cozinha Painel Administrativo

O **backend será o núcleo do sistema**. Ele controlará pedidos, pagamentos, status, usuários, dispositivos, permissões e comunicação entre os módulos.

O frontend poderá ser inicialmente uma aplicação web/PWA e, futuramente, poderá rodar em Android WebView, aplicativo nativo ou modo kiosk.

4. Objetivos do Produto

Objetivo principal

Criar uma solução funcional de autoatendimento para fast food, permitindo que o cliente realize pedidos de forma independente e que o restaurante controle pagamento, preparo, retirada e cardápio.

Objetivos específicos

- Criar um cardápio digital interativo.
- Permitir montagem de pedidos.
- Controlar carrinho de compras.
- Registrar pedidos no backend.
- Separar pedido de pagamento.
- Confirmar pagamentos antes do envio para a cozinha.
- Criar painel do caixa.
- Criar painel da cozinha.
- Criar painel administrativo para gestão do cardápio.
- Proteger a API com autenticação e autorização.
- Cadastrar dispositivos autorizados.
- Preparar o sistema para integrações reais de pagamento.

5. Perfis e Tipos de Acesso

O sistema deverá separar dois tipos principais de acesso:

1. Usuários humanos

2. Dispositivos cadastrados

Essa separação é importante porque um totem, um caixa ou uma tela de cozinha não devem ser tratados exatamente como um usuário comum.

5.1. Usuários Humanos

Usuários humanos são pessoas autorizadas a acessar áreas internas do sistema.

Perfil	Descrição	Exemplos de ações
SUPER_ADMIN	Usuário da empresa desenvolvedora	Gerenciar restaurantes, dispositivos, cardápios e configurações gerais
ADMIN_RESTAURANTE	Responsável pelo restaurante	Gerenciar cardápio, preços, categorias e disponibilidade
OPERADOR_CAIXA	Funcionário do caixa	Confirmar pagamento, cancelar pedidos pendentes, consultar pedidos
OPERADOR_COZINHA	Funcionário da cozinha	Visualizar pedidos pagos e alterar status de preparo

Observação importante

O usuário **SUPER_ADMIN** representa a empresa responsável pelo sistema. Ele permite que os desenvolvedores ou responsáveis pelo produto realizem ajustes no cardápio, produtos, destaques e configurações sem precisar alterar o código da API.

5.2. Dispositivos Cadastrados

Dispositivos são equipamentos autorizados a acessar a API.

Exemplos:

TOTEM_01

Tipo: TOTEM

Restaurante: Loja Centro

Status: Ativo

CAIXA_01

Tipo: CAIXA

Restaurante: Loja Centro

Status: Ativo

COZINHA_01

Tipo: COZINHA

Restaurante: Loja Centro

Status: Ativo

Cada dispositivo deve possuir uma credencial própria e permissões específicas.

Dispositivo	Permissões principais
Totem	Ver cardápio, criar pedido, iniciar pagamento, consultar status
Caixa	Ver pedidos pendentes, confirmar pagamento, cancelar pedido pendente
Cozinha	Ver pedidos pagos, alterar status de preparo

6. Princípio Geral de Segurança

A API não deve ficar aberta para qualquer pessoa ou equipamento da rede.

Toda requisição relevante deve responder às seguintes perguntas:

Quem está acessando?

É usuário ou dispositivo?

Qual restaurante está vinculado?

Qual perfil ou tipo de dispositivo possui?

Essa entidade tem permissão para executar essa ação?

A segurança não deve depender apenas de IP fixo. O IP pode ser usado como camada adicional, mas a proteção principal deve ser feita com:

- Autenticação.
- Autorização.
- Perfis.
- Permissões por endpoint.
- Dispositivos cadastrados.
- Tokens revogáveis.
- Auditoria.

7. Controle por IP

Restringir o acesso por IP pode ser útil, principalmente em cenários de rede local ou instalação em restaurante.

No entanto, **não é recomendado usar apenas IP como proteção principal.**

Modelo fraco

Apenas IP permitido

Modelo recomendado

IP permitido

+ Token válido

+ Dispositivo cadastrado

+ Permissão por função

+ Auditoria

O IP deve ser tratado como uma camada extra de segurança, e não como a única barreira de proteção.

8. Sessão Longa para Totem, Caixa e Cozinha

Não é recomendado criar sessão literalmente infinita.

A melhor solução é criar uma sessão longa, renovável e revogável.

Estratégia recomendada

1. Dispositivo é cadastrado pelo administrador
2. Dispositivo recebe uma credencial própria
3. API emite um access token de curta duração
4. API emite um refresh token de longa duração
5. Dispositivo renova a sessão automaticamente
6. Administrador pode revogar o dispositivo quando necessário

Com isso, o totem pode funcionar por longos períodos sem exigir login manual frequente, mas ainda pode ser bloqueado caso seja roubado, substituído ou comprometido.

9. Ativação de Dispositivo

Para facilitar a instalação, o sistema pode usar um fluxo de ativação.

Fluxo sugerido

Administrador acessa o painel



Cadastra novo dispositivo



Sistema gera código de ativação



Responsável informa o código no tablet/totem



Dispositivo fica vinculado ao restaurante



API libera token específico para aquele dispositivo

Exemplo

Código de ativação: ABC-123-XYZ

Dispositivo: TOTEM_01

Restaurante: Loja Centro

Tipo: TOTEM

Status: Ativo

Depois da ativação, o dispositivo passa a acessar somente os endpoints permitidos para o seu tipo.

10. Módulos do Sistema

10.1. Totem do Cliente

Interface utilizada pelo cliente final.

Funcionalidades

- Tela inicial de boas-vindas.
- Escolha entre comer no local ou levar.
- Listagem de categorias.
- Listagem de produtos.
- Detalhe do produto.
- Seleção de adicionais.
- Remoção de ingredientes, se aplicável.
- Carrinho.
- Identificação do cliente.
- Escolha de forma de pagamento.
- Exibição do status do pedido.
- Exibição da senha ou comprovante.

Permissões do dispositivo Totem

Ver cardápio

Criar pedido

Iniciar pagamento

Consultar status do próprio pedido

O totem não deve ter permissão para:

Alterar produtos

Confirmar pagamento em dinheiro

Cancelar pedidos de outros módulos

Acessar dados administrativos

Gerenciar usuários

10.2. Backend Java

Responsável por centralizar as regras de negócio.

Responsabilidades

- Gerenciar produtos e categorias.
- Criar pedidos.
- Calcular total do pedido.
- Registrar pagamentos.
- Controlar status do pedido.
- Controlar status do pagamento.
- Enviar pedidos pagos para a cozinha.
- Receber confirmação de pagamento.
- Validar autenticação.
- Validar autorização.
- Controlar dispositivos autorizados.
- Registrar auditoria.
- Disponibilizar APIs para totem, caixa, cozinha e administração.

Stack recomendada

Java 21

Spring Boot

Spring Web

Spring Data JPA

Spring Security

PostgreSQL

Bean Validation

Flyway

Swagger/OpenAPI

JWT

BCrypt

10.3. Painel do Caixa

Utilizado por funcionários para controlar pagamentos pendentes.

Funcionalidades

- Ver pedidos aguardando pagamento.
- Confirmar pagamento em dinheiro.
- Confirmar pagamento manual externo, se necessário.
- Cancelar pedido não pago.
- Reimprimir comprovante.
- Consultar histórico de pedidos.

Permissões

DEVICE_CAIXA

OPERADOR_CAIXA

SUPER_ADMIN

10.4. Painel da Cozinha

Utilizado pela equipe de produção.

Funcionalidades

- Ver pedidos pagos.
- Ver itens do pedido.
- Ver adicionais e observações.
- Alterar status para em preparo.
- Alterar status para pronto.
- Organizar fila de preparo.

Regra importante

A cozinha só deve receber pedidos com pagamento confirmado, exceto se houver uma regra manual autorizada futuramente.

Permissões

DEVICE_COZINHA

OPERADOR_COZINHA

SUPER_ADMIN

10.5. Painel Administrativo

Utilizado para gestão do cardápio, usuários, dispositivos e configurações.

Funcionalidades de cardápio

- Cadastrar categorias.
- Cadastrar produtos.
- Editar produtos.
- Remover ou desativar produtos.
- Definir preços.
- Cadastrar adicionais.
- Controlar disponibilidade.
- Definir produtos em destaque.
- Criar recomendações da casa.
- Alterar ordem de exibição.

Funcionalidades de segurança

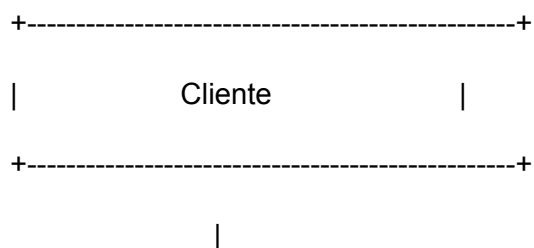
- Criar usuários.
- Definir perfis.
- Cadastrar dispositivos.
- Revogar dispositivos.
- Consultar auditoria.
- Gerenciar permissões básicas.

Permissões

SUPER_ADMIN

ADMIN_RESTAURANTE

11. Arquitetura Geral Recomendada



v

+-----+

| Totem / Tablet / PWA |

| Interface touch para autoatendimento |

+-----+

|

| Token de dispositivo

| REST API / WebSocket futuro

v

+-----+

| Backend Java / Spring Boot |

| Pedidos | Produtos | Pagamentos | Segurança |

| Usuários | Dispositivos | Auditoria |

+-----+

|

v

+-----+

| Banco de Dados |

| Produtos | Pedidos | Pagamentos | Usuários |

| Dispositivos | Tokens | Auditoria |

+-----+

|

|

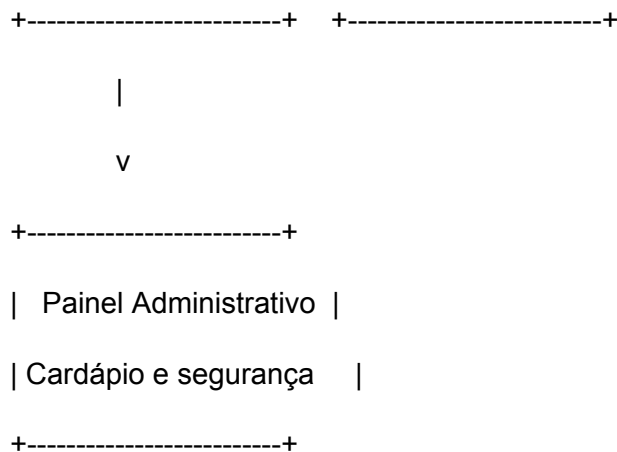
v

v

+-----+ +-----+

| Painel do Caixa | | Painel da Cozinha |

| Pagamentos e conferência | | Preparo e status pedido |



12. Modelo de Comunicação

12.1. Comunicação interna

A comunicação entre totem, caixa e cozinha pode ocorrer em rede local ou pela nuvem.

Opção local

Tablet / Totem

↓ Wi-Fi local

Computador com backend

↓

Banco de dados local

↓

Caixa e cozinha

Opção nuvem

Tablet / Totem

↓ Internet

Backend em nuvem

↓

Banco de dados em nuvem

↓

Caixa e cozinha

Opção recomendada para o início

Para o MVP, recomenda-se iniciar com backend local ou ambiente simples em nuvem, evitando arquitetura híbrida complexa logo no começo.

12.2. Comunicação com pagamento

O pagamento deve ser tratado por um módulo separado.

Pedido

↓

PagamentoService

↓

PaymentProvider

↓

Pix / Cartão / TEF / Maquininha / Fake Provider

13. Estratégia de Pagamento

13.1. Conceito principal

O pedido e o pagamento devem ser controlados separadamente.

Pedido criado

↓

Pagamento pendente



Pagamento confirmado



Pedido enviado para cozinha

A regra central é:

Nenhum pedido deve ser enviado para a cozinha antes da confirmação do pagamento.

13.2. Pagamento via Pix

O Pix é uma das melhores opções para a primeira integração real.

Fluxo Pix

Cliente cria pedido



Seleciona Pix



Backend gera cobrança Pix



Totem exibe QR Code



Cliente paga pelo app do banco



Provedor confirma pagamento



Backend atualiza pagamento como autorizado

↓

Pedido vai para cozinha

Dados importantes para Pix

- Valor do pedido.
 - Identificador da cobrança.
 - QR Code.
 - Status do pagamento.
 - Webhook de confirmação.
 - Horário de expiração.
-

13.3. Pagamento com cartão

O cartão pode ser implementado de formas diferentes.

Modelo A — Cartão simulado no MVP

Cliente escolhe cartão

↓

Sistema simula aprovação

↓

Pedido é marcado como pago

↓

Pedido vai para cozinha

Modelo B — Maquininha externa com confirmação manual

Cliente escolhe cartão

↓

Pedido fica aguardando confirmação

↓

Funcionário cobra na maquininha

↓

Funcionário confirma no painel do caixa

↓

Pedido vai para cozinha

Modelo C — SmartPOS ou SDK Android

Cliente escolhe cartão

↓

Aplicação chama SDK da adquirente

↓

Cliente aproxima ou insere o cartão

↓

SDK retorna aprovado ou recusado

↓

Backend atualiza o pagamento

↓

Pedido vai para cozinha

Modelo D — TEF / Pin Pad

Totem cria pedido

↓

Backend solicita pagamento ao módulo TEF/PDV

↓

Pin Pad realiza transação

↓

TEF retorna resultado

↓

Backend libera ou bloqueia pedido

13.4. Pagamento em dinheiro

O pagamento em dinheiro deve ficar pendente até confirmação do caixa.

Fluxo recomendado

Cliente cria pedido

↓

Seleciona dinheiro

↓

Pedido fica AGUARDANDO_PAGAMENTO_DINHEIRO

↓

Pedido aparece no painel do caixa

↓

Cliente paga no caixa

↓

Funcionário confirma pagamento

↓

Pedido vai para cozinha

14. Camada de Pagamento Desacoplada

Para evitar dependência de uma única empresa de pagamento, recomenda-se criar uma interface de provedor de pagamento.

Interface conceitual

```
public interface PaymentProvider {  
  
    PaymentResponse iniciarPagamento(PaymentRequest request);  
  
    PaymentStatus consultarStatus(String externalPaymentId);  
  
    void cancelarPagamento(String externalPaymentId);  
  
}
```

Implementações possíveis

FakePaymentProvider

PixPaymentProvider

StonePaymentProvider

PagBankPaymentProvider

MercadoPagoProvider

TefPaymentProvider

Benefício

No início, o sistema usa **FakePaymentProvider**. Futuramente, é possível trocar por **PixPaymentProvider** ou outro provedor real sem reescrever todo o sistema.

15. Status do Pedido

Status	Significado
CRIADO	Pedido foi criado no sistema
AGUARDANDO_PAGAMENTO	Pedido aguarda pagamento eletrônico

AGUARDANDO_PAGAMENTO_DINHEIRO	Pedido aguarda pagamento em dinheiro no caixa
PAGO	Pagamento confirmado
ENVIADO_PARA_COZINHA	Pedido liberado para preparo
EM_PREPARO	Cozinha iniciou o preparo
PRONTO	Pedido pronto para retirada
RETIRADO	Cliente retirou o pedido
CANCELADO	Pedido cancelado
EXPIRADO	Pedido não pago dentro do prazo

Enum sugerido

```
public enum StatusPedido {  
    CRIADO,  
    AGUARDANDO_PAGAMENTO,  
    AGUARDANDO_PAGAMENTO_DINHEIRO,  
    PAGO,  
    ENVIADO_PARA_COZINHA,  
    EM_PREPARO,  
    PRONTO,  
    RETIRADO,  
    CANCELADO,  
    EXPIRADO  
}
```

```
CANCELADO,  
EXPIRADO  
}
```

16. Status do Pagamento

Status	Significado
PENDENTE	Pagamento iniciado, mas ainda não confirmado
AUTORIZADO	Pagamento aprovado
RECUSADO	Pagamento recusado
CANCELADO	Pagamento cancelado
ESTORNADO	Pagamento devolvido

Enum sugerido

```
public enum StatusPagamento {  
    PENDENTE,  
    AUTORIZADO,  
    RECUSADO,  
    CANCELADO,  
    ESTORNADO  
}
```

17. Formas de Pagamento

```
public enum FormaPagamento {  
  
    PIX,  
  
    CARTAO_CREDITO,  
  
    CARTAO_DEBITO,  
  
    DINHEIRO  
  
}
```

18. Tipos de Dispositivo

```
public enum TipoDispositivo {  
  
    TOTEM,  
  
    CAIXA,  
  
    COZINHA,  
  
    ADMINISTRACAO  
  
}
```

19. Perfis de Usuário

```
public enum PerfilUsuario {  
  
    SUPER_ADMIN,  
  
    ADMIN_RESTAURANTE,  
  
    OPERADOR_CAIXA,  
  
}
```

```
OPERADOR_COZINHA  
}
```

20. Entidades Principais do Backend

20.1. Restaurante

Representa o estabelecimento que utilizará o sistema.

```
id  
nome  
cnpj  
endereco  
ativo  
criadoEm  
atualizadoEm
```

20.2. Produto

Representa um item vendido no cardápio.

```
id  
restaurantId  
nome  
descricao  
preco  
imagemUrl  
disponivel
```


destaque
recomendado
ordemExibicao
categoriald
criadoEm
atualizadoEm

20.3. Categoria

Agrupa produtos do cardápio.

id
restauranteld
nome
descricao
ordemExibicao
ativa

20.4. Pedido

Representa o pedido realizado pelo cliente.

id
restauranteld
numeroPedido
clienteNome
tipoConsumo

statusPedido

valorTotal

dispositivoOrigemId

criadoEm

atualizadoEm

20.5. ItemPedido

Representa cada produto dentro de um pedido.

id

pedidoId

produtoId

nomeProduto

quantidade

precoUnitario

subtotal

observacao

20.6. Pagamento

Representa o pagamento vinculado ao pedido.

id

pedidoId

formaPagamento

statusPagamento

valor
paymentProvider
externalPaymentId
qrCodePix
criadoEm
pagoEm
canceladoEm

20.7. Usuario

Representa um usuário humano autorizado.

id
restaurantId
nome
email
senhaHash
perfil
ativo
criadoEm
atualizadoEm

20.8. Dispositivo

Representa um equipamento autorizado a acessar a API.

id

restaurantId
nome
codigoIdentificacao
tipoDispositivo
ativo
ultimoAcesso
criadoEm
atualizadoEm

20.9. RefreshToken

Representa um token de renovação de sessão.

id
usuarioId
dispositivoId
tokenHash
expiraEm
revogado
criadoEm
revogadoEm

20.10. HistoricoStatusPedido

Registra a evolução do pedido.

id

pedidoId

statusAnterior

statusNovo

dataAlteracao

alteradoPorUsuarioId

alteradoPorDispositivoId

observacao

20.11. Auditoria

Registra ações relevantes no sistema.

id

restaurantId

usuarioId

dispositivoId

acao

entidadeAfetada

entidadeId

dataHora

ipOrigem

detalhes

21. Organização Recomendada do Backend

src/main/java/com/projeto/totem

└─ controller
| └─ admin
| └─ auth
| └─ caixa
| └─ cozinha
| └─ totem
└─ service
└─ repository
└─ entity
└─ dto
└─ enums
└─ exception
└─ config
└─ security
└─ integration
| └─ payment
└─ mapper

Responsabilidade das camadas

Camada	Responsabilidade
controller	Receber requisições HTTP
service	Aplicar regras de negócio
repository	Acessar o banco de dados

entity	Representar tabelas
dto	Transportar dados entre API e cliente
enums	Status, formas de pagamento e tipos fixos
exception	Tratamento padronizado de erros
config	Configurações gerais
security	Autenticação, autorização, JWT e filtros
integration	Integrações externas
mapper	Conversão entre entity e DTO

22. Endpoints Iniciais

22.1. Autenticação e Segurança

POST /api/auth/login

POST /api/auth/refresh

POST /api/auth/logout

POST /api/auth/dispositivos/ativar

22.2. Totem

GET /api/totem/cardapio

POST /api/totem/pedidos

GET /api/totem/pedidos/{id}

POST /api/totem/pedidos/{id}/pagamento

Permissão sugerida:

DEVICE_TOTEM

22.3. Caixa

GET /api/caixa/pedidos/pendentes

POST /api/caixa/pedidos/{id}/confirmar-pagamento

POST /api/caixa/pedidos/{id}/cancelar

Permissão sugerida:

DEVICE_CAIXA

OPERADOR_CAIXA

SUPER_ADMIN

22.4. Cozinha

GET /api/cozinha/pedidos

PATCH /api/cozinha/pedidos/{id}/status

Permissão sugerida:

DEVICE_COZINHA

OPERADOR_COZINHA

SUPER_ADMIN

22.5. Administração de Cardápio

POST /api/admin/categorias

GET /api/admin/categorias

PUT /api/admin/categorias/{id}

DELETE /api/admin/categorias/{id}

POST /api/admin/produtos

GET /api/admin/produtos

PUT /api/admin/produtos/{id}

DELETE /api/admin/produtos/{id}

PATCH /api/admin/produtos/{id}/disponibilidade

PATCH /api/admin/produtos/{id}/destaque

Permissão sugerida:

SUPER_ADMIN

ADMIN_RESTAURANTE

22.6. Administração de Dispositivos

POST /api/admin/dispositivos

GET /api/admin/dispositivos

PATCH /api/admin/dispositivos/{id}/revogar

PATCH /api/admin/dispositivos/{id}/ativar

Permissão sugerida:

SUPER_ADMIN

ADMIN_RESTAURANTE

22.7. Administração de Usuários

POST /api/admin/usuarios

GET /api/admin/usuarios

PUT /api/admin/usuarios/{id}

PATCH /api/admin/usuarios/{id}/desativar

Permissão sugerida:

SUPER_ADMIN

ADMIN_RESTAURANTE

22.8. Webhooks futuros

POST /api/webhooks/pix

POST /api/webhooks/pagamentos

Esses endpoints poderão ser utilizados futuramente para receber confirmação automática de pagamento de provedores externos.

23. Fluxos Principais do Sistema

23.1. Fluxo de ativação do totem

Administrador cadastra dispositivo



Sistema gera código de ativação



Responsável informa código no tablet



Backend valida código



Dispositivo é ativado



Totem recebe credencial/token



Totem passa a acessar apenas endpoints permitidos

23.2. Fluxo de pedido com Pix

Cliente acessa totem



Seleciona produtos



Confere carrinho



Escolhe Pix



Backend cria pedido e pagamento pendente



Totem exibe QR Code



Cliente paga



Backend recebe confirmação



Pedido muda para PAGO



Pedido é enviado para cozinha

23.3. Fluxo de pedido com cartão simulado

Cliente acessa totem



Seleciona produtos



Escolhe cartão



Sistema simula aprovação



Pagamento fica AUTORIZADO



Pedido muda para PAGO



Pedido vai para cozinha

23.4. Fluxo de pedido com dinheiro

Cliente acessa totem



Seleciona produtos



Escolhe dinheiro



Pedido fica aguardando pagamento no caixa



Cliente paga no caixa



Funcionário confirma pagamento



Pedido vai para cozinha

23.5. Fluxo da cozinha

Pedido pago aparece na cozinha



Funcionário inicia preparo



Pedido muda para EM_PREPARO



Pedido fica pronto



Status muda para PRONTO



Cliente retira



Status muda para RETIRADO

23.6. Fluxo de alteração de cardápio

Administrador acessa painel



Realiza login



Cria, edita ou desativa produto



Sistema registra auditoria



Totem passa a exibir cardápio atualizado

24. Comprovante, Comanda e Nota Fiscal

Na primeira versão, recomenda-se emitir apenas um **comprovante de pedido**, **comanda** ou **senha de retirada**.

Evitar chamar de nota fiscal no MVP, pois nota fiscal real envolve obrigações fiscais, integração com sistemas fiscais e autorização governamental.

Exemplo de comprovante

PEDIDO #1024

Cliente: Fábio

Itens:

1x Combo X-Burger

1x Batata Média

1x Refrigerante

Total: R\$ 39,90

Pagamento: Pix

Status: Pago

Retirada no balcão

25. Controle do Tablet e Modo Totem

A tela do cliente pode ser executada de diferentes formas.

Opção	Descrição	Recomendação
Web/PWA	Aplicação web em tela cheia	Melhor para MVP

Android WebView	App Android abrindo a aplicação web	Boa evolução futura
Android nativo	App feito em Java/Kotlin	Útil para integração com SDK de maquininha
Kiosk Mode	Tablet travado em um app	Ideal para operação real

Recomendação inicial

Começar com uma aplicação web/PWA e evoluir para Android/WebView apenas se a integração com maquininha exigir.

26. MVP Recomendado

A primeira versão deve validar o fluxo completo sem depender de hardware externo complexo.

Funcionalidades do MVP

- Cadastro de restaurante.
- Login administrativo.
- Cadastro de categorias.
- Cadastro de produtos.
- Produtos em destaque ou recomendados.
- Listagem do cardápio no totem.
- Carrinho.
- Criação do pedido.
- Pagamento simulado.
- Dinheiro com confirmação no caixa.
- Cadastro básico de dispositivos.
- Token de acesso para dispositivos.
- Painel do caixa.
- Painel da cozinha.
- Atualização de status do pedido.
- Comprovante de pedido.
- Auditoria básica para alterações de cardápio.

O que deixar para depois

- Pix real.
 - Integração com maquininha.
 - TEF.
 - Impressora térmica.
 - Nota fiscal.
 - Relatórios avançados.
 - Controle de estoque.
 - mTLS.
 - 2FA.
-

27. Roadmap de Desenvolvimento

Fase 1 — Planejamento e estrutura

- Definir entidades.
- Definir regras de negócio.
- Criar projeto backend.
- Configurar banco.
- Criar migrations.
- Configurar Swagger.

Fase 2 — Segurança inicial

- Configurar Spring Security.
- Criar entidade Usuario.
- Implementar login administrativo.
- Implementar JWT.
- Criptografar senhas com BCrypt.
- Criar perfis de usuário.
- Proteger endpoints administrativos.

Fase 3 — Cardápio

- Criar categorias.
- Criar produtos.
- Permitir destaque/recomendação.
- Controlar disponibilidade.
- Listar cardápio para o totem.

Fase 4 — Dispositivos

- Criar entidade Dispositivo.
- Cadastrar Totem, Caixa e Cozinha.

- Implementar ativação de dispositivo.
- Criar permissões por tipo de dispositivo.
- Criar refresh token revogável.

Fase 5 — Pedido

- Criar pedido.
- Adicionar itens.
- Calcular valor total.
- Criar número do pedido.
- Registrar histórico de status.

Fase 6 — Pagamento simulado

- Criar entidade Pagamento.
- Criar `FakePaymentProvider`.
- Simular Pix.
- Simular cartão.
- Criar confirmação manual para dinheiro.

Fase 7 — Painel do caixa

- Listar pedidos pendentes.
- Confirmar pagamento em dinheiro.
- Cancelar pedido.

Fase 8 — Painel da cozinha

- Listar pedidos pagos.
- Mudar status para em preparo.
- Mudar status para pronto.
- Finalizar retirada.

Fase 9 — Interface de totem

- Criar layout touch friendly.
- Criar fluxo de compra.
- Criar tela de pagamento.
- Criar tela de confirmação.
- Criar timeout de inatividade.

Fase 10 — Evoluções reais

- Pix real.

- Webhook de pagamento.
 - Integração com maquininha.
 - Impressão de comanda.
 - Modo kiosk.
 - Relatórios.
-

28. Decisões Técnicas Iniciais

Decisão	Escolha recomendada	Motivo
Backend	Java + Spring Boot	Stack robusta e adequada para APIs
Segurança	Spring Security + JWT	Proteção da API e controle de acesso
Senhas	BCrypt	Armazenamento seguro de senha
Banco	PostgreSQL	Confiável, gratuito e profissional
Frontend	Angular ou React	Bom para web/PWA e painéis
Pagamento inicial	Simulado	Evita travar o MVP por hardware externo
Pagamento futuro	Pix, SDK, TEF ou SmartPOS	Permite evolução comercial
Controle de dispositivos	Token por dispositivo	Mais seguro do que sessão infinita
Tablet	PWA inicialmente	Mais simples para validar

Cozinha	Painel web	Fácil de implementar e testar
Caixa	Painel web	Necessário para dinheiro e confirmação manual
Administração	Painel administrativo	Permite alterar cardápio sem mudar código

29. Principais Regras de Negócio

1. Um pedido deve possuir pelo menos um item.
 2. Um pedido deve ter valor total maior que zero.
 3. Um pedido só pode ir para cozinha após pagamento confirmado.
 4. Pedido em dinheiro deve aguardar confirmação do caixa.
 5. Pedido com pagamento recusado não deve ser enviado para cozinha.
 6. Pedido não pago dentro de determinado tempo pode expirar.
 7. Produto indisponível não deve aparecer para compra no totem.
 8. Alterações no pedido devem ser bloqueadas após pagamento confirmado.
 9. Todo status importante deve ser registrado em histórico.
 10. O comprovante deve exibir número do pedido, itens, valor, forma de pagamento e status.
 11. Totem só pode criar pedidos e consultar informações permitidas.
 12. Caixa só pode confirmar ou cancelar pedidos dentro do seu escopo.
 13. Cozinha só pode visualizar pedidos pagos e alterar status de preparo.
 14. Administração pode alterar cardápio sem necessidade de alterar código.
 15. Toda alteração administrativa relevante deve gerar registro de auditoria.
 16. Dispositivo revogado não pode continuar acessando a API.
 17. Segurança não deve depender apenas do IP da máquina.
-

30. Riscos e Pontos de Atenção

Risco

Impacto

Mitigação

Integração com maquininha ser complexa	Pode atrasar o projeto	Começar com pagamento simulado
Pix exigir provedor externo	Depende de API/gateway	Preparar camada PaymentProvider
Internet cair	Pode afetar pagamentos	Avaliar operação local futura
Pedido ir para cozinha sem pagamento	Prejuízo operacional	Bloquear envio até pagamento confirmado
Tablet permitir sair do sistema	Má experiência/risco de uso indevido	Usar PWA fullscreen ou kiosk mode
Nota fiscal real ser complexa	Escopo fiscal elevado	Começar com comprovante interno
API ficar aberta na rede	Risco de uso indevido	Autenticação, autorização e token por dispositivo
Sessão infinita ser comprometida	Acesso indevido prolongado	Usar refresh token revogável
Funcionário alterar cardápio sem permissão	Erros operacionais	Perfis e permissões administrativas
Produto ser alterado sem rastreio	Falta de controle	Auditoria de alterações

31. Perguntas em Aberto

Estas decisões podem ser tomadas antes ou durante o desenvolvimento:

- O sistema será para uso local, nuvem ou ambos?
- O primeiro frontend será Angular ou React?
- O pagamento Pix será integrado em qual fase?
- O cartão será inicialmente simulado ou confirmado manualmente?
- Haverá cadastro de usuário funcionário desde o MVP?
- O pedido poderá ser alterado depois de criado?
- Produtos terão adicionais obrigatórios e opcionais?
- Haverá controle de estoque?
- Haverá impressão de comanda?
- O sistema terá login administrativo desde a primeira versão?
- Cada restaurante terá um ou vários totens?
- O sistema será multi-restaurante desde o início?
- O administrador do restaurante poderá alterar tudo ou apenas cardápio?
- O token do dispositivo terá validade de quantos dias?
- O IP será usado como camada adicional de segurança?

32. Versão Oficial da Ideia

O sistema será uma solução de autoatendimento para fast foods, composta por um totem/tablet para realização de pedidos, um backend em Java responsável pelas regras de negócio, um painel de caixa para confirmação de pagamentos pendentes, um painel de cozinha para acompanhamento da produção e um painel administrativo para gestão do cardápio, usuários e dispositivos.

A comunicação entre totem, caixa e cozinha poderá ocorrer em rede local ou ambiente em nuvem. A API será protegida por autenticação, autorização, perfis de acesso e dispositivos cadastrados, evitando que qualquer máquina ou usuário não autorizado consiga acessar ou modificar dados do sistema.

O pagamento será tratado por um módulo separado, permitindo iniciar com pagamentos simulados e evoluir futuramente para Pix via API, integração com SmartPOS, SDK de maquininhas ou TEF.

O pedido só será enviado para a cozinha após a confirmação do pagamento, garantindo segurança operacional e controle do fluxo.

33. Conclusão

O projeto é viável, tem boa aplicação prática e pode se tornar um excelente produto ou projeto de portfólio.

A arquitetura recomendada deve considerar desde o início três pilares:

Pedido

Pagamento

Segurança

Além disso, o sistema deve permitir administração dinâmica do cardápio, evitando que alterações de produtos, preços, imagens ou destaques dependam de mudanças no código.

A melhor estratégia é iniciar com um MVP funcional, com segurança básica, login administrativo, controle de dispositivos, pagamento simulado, painel do caixa e painel da cozinha.

A ordem recomendada é:

1. Segurança inicial
2. Cardápio administrativo
3. Cadastro de dispositivos
4. Pedido
5. Pagamento simulado
6. Caixa
7. Cozinha
8. Interface do totem
9. Pix real
10. Integração com maquininha

Essa abordagem permite validar a ideia rapidamente, reduzir riscos técnicos e manter o sistema preparado para crescimento futuro.